ADEKUNLE AJASIN UNIVERSITY, AKUNGBA-AKOKO · DEPARTMENT OF COMPUTER SCIENCE
CSC426 – WEEKLY DEV, WEEKEND 3 (01/07/26) · Lecturer: Dr. O. O. Ajayi

IRO – ISSUES REPORTING OUTLET

Name: Joshua Omolafe	**Matric No.:** 220404244	**GitHub:** github.com/Joshua989/Weekend-3-01-07-26-

Contents: 1. SRS (pp. 1–2) · 2. Use-Case Diagram (p. 3) · 3. Ishikawa Diagram (p. 3) · 4. Object Listing (p. 4) · 5. Code Listing (pp. 4–5)

1. SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1.1 Purpose and Scope

IRO is a web-based outlet through which AAUA students and indigenes of Akungba, Iwaro, Etioro, Ayegunle and neighbouring communities report issues that affect town–gown relations (rent, security, land, harassment, festivals, sanitation, utilities, transport, prices). Today such issues go through scattered informal channels, many are never reported for fear of victimisation, and reporters rarely learn the outcome. IRO lets anyone submit a report (anonymously if they wish), gives each report a ticket ID for tracking, and gives an IRO desk officer a dashboard to review, escalate and resolve reports with a logged history. SMS alerts, file uploads, a server database and Yoruba translation are outside the scope of this version.

1.2 Users

User	Role
Student / Indigene (Reporter)	Submits and tracks reports. Students may give a matric number; indigenes select their community.
IRO Desk Officer (Admin)	Logs in, views statistics, filters reports, updates status, escalates and deletes reports.
Escalation Authority	Students' Affairs, community leaders (Oba/Baale) or police; receives escalated cases offline.

1.3 Major Components and their Interaction

Component	File(s)	Responsibility
Pages (UI)	index, report, track, admin .html; style.css	Home & guidelines, report form, tracking page, admin desk.
ReportForm	js/report.js	Shows student/indigene fields, validates input, calls IssueStore.create(), shows ticket ID.
Tracker	js/track.js	Calls IssueStore.findById(); shows status and history without personal data.
AdminDesk	js/admin.js	Login, statistics, search/filter, detail view, status update, escalation, deletion.
IssueStore	js/store.js	The only data access point: create, find, update status, remove, stats, ticket IDs.
Storage	Browser localStorage	Holds all issues as JSON (to be replaced by a server database later).

Interaction: Reporter → report.html → ReportForm.validate() → IssueStore.create() → localStorage → ticket ID shown. Reporter → track.html → Tracker → IssueStore.findById() → status + history shown. Desk Officer → admin.html → AdminDesk → IssueStore.updateStatus() (Submitted → Under Review → Escalated → Resolved → Closed, each change logged with time and note) → reporter sees the update when tracking. Escalated cases are forwarded to the Escalation Authority.

1.4 Constraints and Assumptions

Plain HTML5/CSS3/JavaScript, no frameworks; runs in any modern browser with no installation. Data is kept in the browser, so reports stay on the device used. Personal data must be handled in line with the Nigeria Data Protection Act 2023. Assumes the university and community leaders agree to act on escalated reports.

1.5 Functional Requirements

ID	The system shall…
FR-01	display IRO's purpose, the reporting steps, issue categories and an emergency notice on the home page.
FR-02	require the reporter to choose Student, Indigene or Other.
FR-03	show an optional matric-number field for students and a required community list (Akungba, Iwaro, Etioro, Ayegunle, Oke-Oka, Other) for indigenes.
FR-04	allow anonymous reporting; name and contact are then hidden and not stored.
FR-05	capture category, severity (Low/Medium/High/Emergency), location, incident date, title and description.
FR-06	reject invalid input with a clear message (missing fields, future date, short title/description, bad phone/e-mail).
FR-07	create a unique ticket ID (IRO-YY-XXXXX), set status to Submitted and show the ticket with a tracking link.
FR-08	let anyone track a report by ticket ID and see its status and history, without showing personal details.
FR-09	restrict the admin desk to users with the correct passcode.
FR-10	show total, open, emergency and resolved report counts.
FR-11	list reports newest first, with search (ticket, title, location) and filters (status, category, reporter type).
FR-12	let the officer view full details, change status with a note (timestamped history), escalate, and delete after confirmation.

1.6 Non-Functional Requirements

ID	Attribute	Requirement
NFR-01	Performance	Pages usable within 2 s on 3G; search and submit respond within 1 s for 1,000 reports; app under 200 KB.
NFR-02	Confidentiality	Anonymous reports store no identity; personal data is visible only on the admin desk.
NFR-03	Security	Admin requires authentication; all user text is HTML-escaped (XSS protection); ticket IDs are random (32^5 combinations).
NFR-04	Usability	A first-time user can submit a report in under 3 minutes; plain English; labelled inputs; visible focus.
NFR-05	Responsiveness	Works from 320 px phones to desktops without horizontal scrolling.
NFR-06	Reliability	Every status change is logged and never edited; corrupt storage starts an empty list instead of crashing.
NFR-07	Maintainability	All data access goes through IssueStore; lists are configured in one IRO object; code is commented.
NFR-08	Portability	Runs in current Chrome, Firefox, Edge and Safari with no installation.
NFR-09	Legal / ethical	Complies with the NDPA 2023; wording treats students and indigenes equally.

1.7 Traceability to the Ishikawa root causes

Root cause	Addressed by
Fear of victimisation; no confidentiality policy	FR-04, FR-08, NFR-02
No single formal reporting channel; low awareness	FR-01 – FR-07
No tracking or feedback to reporter	FR-07, FR-08, FR-12
Unclear responsibility; weak enforcement	FR-12 (escalation and logged history)
No digital platform; lost records; no trend data	Whole system, FR-10, FR-11, NFR-06

2. USE-CASE DIAGRAM

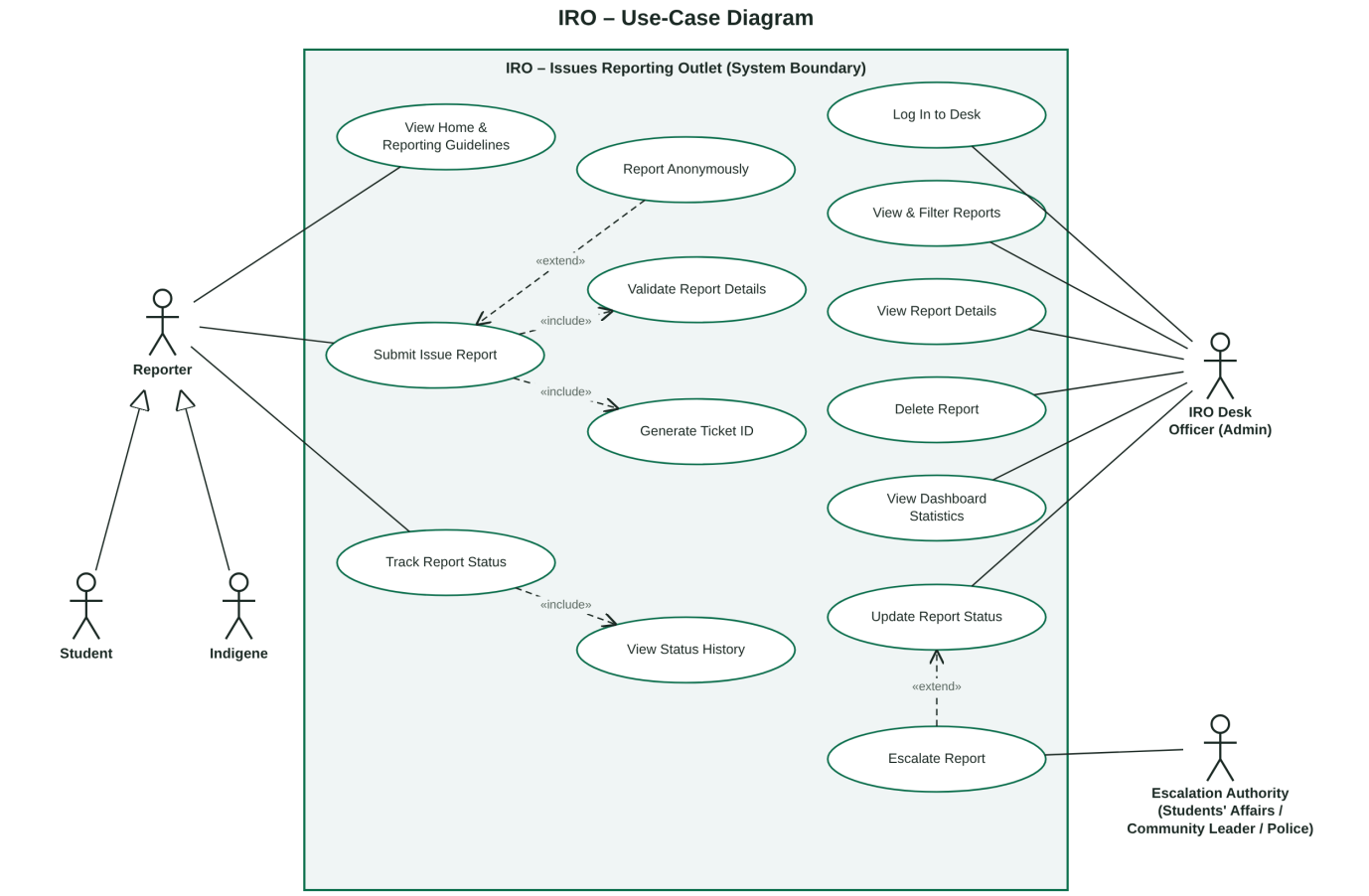


Figure 1: Actors and use cases of IRO (source: docs/use-case/IRO_UseCase.drawio)

3. ISHIKAWA (FISHBONE) PROBLEM ANALYSIS

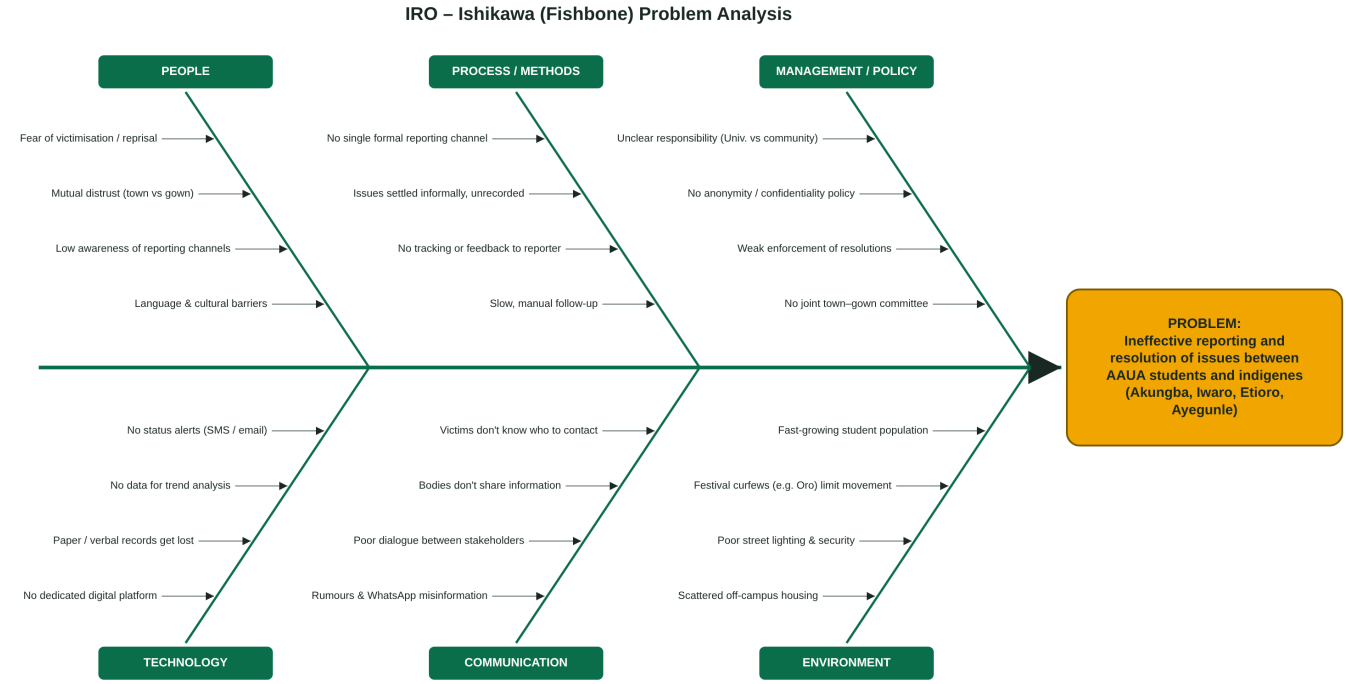


Figure 2: Causes of ineffective issue reporting (source: docs/ishikawa/IRO_Ishikawa.drawio)

4. OBJECT LISTING

Object	File	Properties / methods
`IRO`	store.js	Configuration: STORAGE_KEY, ADMIN_PASSCODE, CATEGORIES, COMMUNITIES, STATUSES, SEVERITIES.
`IssueStore`	store.js	Data layer: getAll(), saveAll(list), generateId(), create(data), findById(id), updateStatus(id, status, note), remove(id), stats().
Helpers	store.js	escapeHtml(str), formatDate(iso), statusBadge(status), fillSelect(select, items, placeholder).
`ReportForm`	report.js	form, el(id), init(), toggleReporterFields(), toggleIdentity(), collect(), validate(d), submit(e).
`Tracker`	track.js	el(id), init(), show(id).
`AdminDesk`	admin.js	SESSION_KEY, selectedId, el(id), init(), login(e), logout(), showDashboard(), filtered(), render(), openDetail(id), saveUpdate(e), deleteSelected().
Issue (record)	localStorage	id, reporterType, anonymous, name, contact, matricNo, community, category, severity, location, incidentDate, title, description, status, createdAt, history[{status, note, at}].

5. CODE LISTING

Core data layer `IssueStore` from `app/js/store.js`, the object every page uses to read and write reports. The full source code (HTML, CSS and JavaScript) is in the GitHub repository above.

```
35 const IssueStore = {
36   /** Return every saved issue (newest first). */
37   getAll() {
38     try {
39       const raw = localStorage.getItem(IRO.STORAGE_KEY);
40       const list = raw ? JSON.parse(raw) : [];
41       return list.sort((a, b) => b.createdAt.localeCompare(a.createdAt));
42     } catch (e) {
43       return [];
44     }
45   },
46
47   /** Persist the full list. */
48   saveAll(list) {
49     localStorage.setItem(IRO.STORAGE_KEY, JSON.stringify(list));
50   },
51
52   /** Create a unique, human-friendly ticket ID such as IRO-26-4F7K2. */
53   generateId() {
54     const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"; // no 0/O/1/I to avoid
   confusion
55     const year = String(new Date().getFullYear()).slice(-2);
56     let id;
57     do {
58       let code = "";
59       for (let i = 0; i < 5; i++) code += chars[Math.floor(Math.random() *
   chars.length)];
60       id = `IRO-${year}-${code}`;
61     } while (this.findById(id));
62     return id;
63   },
64
65   /** Add a new issue and return it. */
66   create(data) {
67     const now = new Date().toISOString();
68     const issue = {
69       id: this.generateId(),
70       reporterType: data.reporterType,          // "Student" | "Indigene" |
   "Other"
71       anonymous: !!data.anonymous,
72       name: data.anonymous ? "" : data.name.trim(),
73       contact: data.anonymous ? "" : data.contact.trim(),
74       matricNo: data.matricNo ? data.matricNo.trim() : "",
```

```
      community: data.community || "",
      category: data.category,
      severity: data.severity,
      location: data.location.trim(),
      incidentDate: data.incidentDate,
      title: data.title.trim(),
      description: data.description.trim(),
      status: "Submitted",
      createdAt: now,
      history: [{ status: "Submitted", note: "Report received by IRO.", at: now }]
    };
    const list = this.getAll();
    list.push(issue);
    this.saveAll(list);
    return issue;
  },

  /** Find one issue by its ticket ID (case-insensitive). */
  findById(id) {
    if (!id) return null;
    const key = id.trim().toUpperCase();
    return this.getAll().find(i => i.id === key) || null;
  },

  /** Change an issue's status and log the change in its history. */
  updateStatus(id, status, note) {
    const list = this.getAll();
    const issue = list.find(i => i.id === id);
    if (!issue) return null;
    issue.status = status;
    issue.history.push({ status, note: note || "", at: new Date().toISOString() });
    this.saveAll(list);
    return issue;
  },

  /** Remove an issue permanently (admin only). */
  remove(id) {
    this.saveAll(this.getAll().filter(i => i.id !== id));
  },

  /** Counts used by the admin dashboard. */
  stats() {
    const list = this.getAll();
    const byStatus = {};
    IRO.STATUSES.forEach(s => (byStatus[s] = 0));
    list.forEach(i => byStatus[i.status]++);
    return {
      total: list.length,
      open: list.length - byStatus["Resolved"] - byStatus["Closed"],
      emergency: list.filter(i => i.severity === "Emergency" && i.status !== "Closed").length,
      byStatus
    };
  }
};
```